# Engineering Principles and Practices

**IITM Explainable Slotbook | CS5013 Mid Evaluation | 11 October 2026**

Nishanth Senthil Kumar EE23B049 | Sparsh Gupta EE23B117

This project applies the course principles as an auditable engineering loop: reviewed prompt, focused implementation, automated verification, documentation, and a coherent Git commit. The system collects synthetic course preferences, maps IITM constraints into UniTime, checks results against CP-SAT, and publishes deterministic evidence for every slot assignment.

## 1 Prompt Engineering

Prompts are versioned engineering inputs, not disposable chat messages. Each substantial task is recorded in prompts/PROMPT_LOG.md with CoSTAR structure, negative constraints, review notes, acceptance checks, and the commit it influenced.

| CoSTAR field | How we apply it |
|---|---|
| **Context** | Name the current architecture, data provenance, relevant files, and known constraints. |
| **Objective** | State one observable outcome, such as a valid fixture or a passing API contract. |
| **Style** | Require small cohesive modules, stable schemas, and maintainable test assertions. |
| **Tone** | Use precise claims; label synthetic data, uncertainty, and heuristic results honestly. |
| **Audience** | Target the administrator, evaluator, student, professor, or future maintainer. |
| **Response** | Specify files, commands, tests, documentation, and acceptance evidence. |

**Review gate.** We check specificity, hidden assumptions, privacy, architecture fit, failure modes, and verifiability before accepting generated work. Prompt revisions and rejected outputs remain in the log so Git records the reasoning, not only the final diff.

**Representative negative prompts.** Do not use real student identities; do not hand-author the final timetable; do not let an LLM decide or invent assignment reasons; do not change CSV contracts silently; do not claim UniTime is globally optimal; and do not publish until validation and CI succeed.

## 2 Structured Debugging

We use reproduce, observe, hypothesize, isolate, correct, test, regress, and document. A failure is first reduced to one command or fixture. Competing hypotheses are written before editing, and the smallest correction is made in the module that owns the behaviour.

| Case | Evidence and correction |
| --- | --- |
| **Objective mismatch** | UniTime reported 12 for a schedule whose domain cost was 20. Inspection showed cpsolver scores time preferences relative to each class minimum. We restored the constant offset and added an independent penalty-equivalence regression test. |
| **Non deterministic tests** | Timetables differed between JVM runs. A randomised map iteration order changed constraint insertion order. Stable pair ordering fixed it; the same iteration cap now reproduces the same timetable. |
| **Apps Script 404** | The Vercel bridge followed a POST ContentService redirect automatically and received HTML. We validated the script.googleusercontent.com redirect and retrieved it with GET; a Node contract test now pins that behaviour. |
| **Weak showcase** | The first demo had zero student clashes, so it failed the presentation goal despite being valid. Six synthetic pair coverage profiles made all 28 elective pairs positive, forcing one explainable minimum weight overlap. |

## 3 Version Control

Commits are coherent verified increments. Prompts are committed with the implementation they influence; secrets, IDE state, build directories, and temporary exports are excluded. Before every commit we inspect the diff and run the nearest tests, then the applicable full suite.

| Commit | Purpose and verification boundary |
| --- | --- |
| **487a534** | Added the showcase dataset, workbook, validator, CI generation, and reviewed prompt. |
| **f4330ee** | Rejected the weak scenario and forced a genuine explainable optimisation trade off. |
| **8269d7f** | Retained solver generated outputs only after UniTime matched CP-SAT objective 9. |

Branch protection is supported by GitHub Actions on every push and pull request. The workflow gives future refactors the same executable acceptance criteria as the original implementation.

# 4 Testing Refactoring and Documentation

Tests target observable guarantees instead of one fragile timetable. Multiple schedules can have equal cost, so assertions check completeness, domains, fixed slots, real time overlaps, labs, rooms, protected pairs, objective recomputation, explanation coverage, and failure codes.

| Evidence | Verified result |
|---|---|
| **Java model** | 52 JUnit tests across parsing, validation, both solvers, exports, generated scenarios, independent cost checks, and brute force small cases. |
| **Portal** | 7 Node API contract tests plus a static validator covering 18 catalog courses, 17 slots, required assets, and CSV headers. |
| **Showcase** | 11 courses, 36 synthetic students, 144 ranked rows, 6 protected pairs, and all 28 elective co demand pairs covered. |
| **Cross solver check** | UniTime produced objective 9 in 10.108 seconds; CP-SAT proved objective 9 optimal in 0.199 seconds. |
| **Continuous integration** | CI compiles Java 17, runs both test suites and validators, executes both solvers, and uploads the generated slotbooks. |

**Refactoring.** TimeGrid is the single definition of overlap; Objective is the single definition of cost; TimetableSolver isolates engines; UniTime imports stay in solver.unitime; and CLI, JavaFX, and the portal depend on stable outputs rather than solver variables. The portal similarly separates static forms, server-side API validation, Apps Script persistence, and publication.

**Documentation.** The repository includes a man-page-style manual with name, synopsis, options, inputs, outputs, exit status, examples, and limitations; an architecture flow; code walkthrough; test plan; verification record; debugging guide; file map; prompt register; and this evaluation runbook.

# Appendix Sample Reviewed Prompts

**Showcase prompt.** Context: the portal exports solver-compatible preferences. Objective: create a compact synthetic case that demonstrates every important constraint and one genuine trade off. Style: evaluator-facing CSVs and a workbook. Audience: CS5013 evaluators. Response: fixture, validator, both solver outputs, CI artifact, and public explanation. Negative constraints: no real IDs, no hand-authored result, no hidden hard-rule violations, and no optimality claim without CP-SAT proof.

**Live change prompt.** Context: add a positive-class-size invariant without changing solver behaviour. Objective: reject class_size less than one with a stable issue code. Style: smallest change in InputValidator plus one focused JUnit case. Audience: maintainers and evaluator. Response: code diff, focused command, full-suite command, and documentation impact. Negative constraints: do not weaken existing validation, edit the CSV parser, or assert an error-message sentence instead of the issue code.